

Run `valgrind` and check the result as: All heap blocks were freed -- no leaks are possible.

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    int x=100;
    if(fork()==0)
    {
        printf("Child Before=%d\n",x);
        x=500;
        printf("Child After=%d\n",x);
    }
    else
    {
        sleep(2);
        printf("Parent=%d\n",x);
    }

    return 0;
}
```

```
eesha@DESKTOP-6R05L78:~$ nano cbca.c
eesha@DESKTOP-6R05L78:~$ gcc cbca.c -o c
eesha@DESKTOP-6R05L78:~$ ./c
Child Before=100
Child After=500
Parent=100
eesha@DESKTOP-6R05L78:~$ |
```

```
eesha@DESKTOP-6R05L78:~$ nano cbca.c
eesha@DESKTOP-6R05L78:~$ gcc -g cbca.c -o c && valgrind ./c
==1655== Memcheck, a memory error detector
==1655== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1655== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1655== Command: ./c
==1655==
Child Before=100
Child After=500
==1656==
==1656== HEAP SUMMARY:
==1656==      in use at exit: 0 bytes in 0 blocks
==1656==    total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==1656==
==1656== All heap blocks were freed -- no leaks are possible
==1656==
==1656== For lists of detected and suppressed errors, rerun with
: -s
==1656== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
Parent=100
==1655==
==1655== HEAP SUMMARY:
==1655==      in use at exit: 0 bytes in 0 blocks
==1655==    total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==1655==
==1655== All heap blocks were freed -- no leaks are possible
==1655==
==1655== For lists of detected and suppressed errors, rerun with
: -s
==1655== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
eesha@DESKTOP-6R05L78:~$ |
```

8. Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a,*b;

    a=(int*)malloc(5*sizeof(int));

    b=(int*)calloc(5,sizeof(int));

    a=(int*)realloc(a,10*sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

```
eesha@DESKTOP-6R05L78:~$ ./m
eesha@DESKTOP-6R05L78:~$ nano mcr.c
eesha@DESKTOP-6R05L78:~$ gcc -g mcr.c -o m && valgrind ./m
==1628== Memcheck, a memory error detector
==1628== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Sewar
d et al.
==1628== Using Valgrind-3.26.0 and LibVEX; rerun with -h for cop
yright info
==1628== Command: ./m
==1628==
==1628==
==1628== HEAP SUMMARY:
==1628==       in use at exit: 0 bytes in 0 blocks
==1628==    total heap usage: 3 allocs, 3 frees, 80 bytes allocat
ed
==1628==
==1628== All heap blocks were freed -- no leaks are possible
==1628==
==1628== For lists of detected and suppressed errors, rerun with
: -s
==1628== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
eesha@DESKTOP-6R05L78:~$ |
```

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int global=100;
static int s=50;

int main()
{
    int stack=10;
    int *heap=(int*)malloc(sizeof(int));

    printf("Code Segment : %p\n",main);
    printf("Global Segment : %p\n",&global);
    printf("Static Segment : %p\n",&s);
    printf("Heap Segment : %p\n",heap);
    printf("Stack Segment : %p\n",&stack);
    free(heap);
    return 0;
}
```

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

PS
cat /proc/PID/maps

```
eesha@DESKTOP-6R05L78:~$ nano memseg.c
eesha@DESKTOP-6R05L78:~$ gcc memseg.c -o ms
eesha@DESKTOP-6R05L78:~$ ./ms
Code Segment : 0x56460d24c1a9
Global Segment: 0x56460d24f010
Static Segment: 0x56460d24f014
Heap Segment : 0x56461094a010
Stack Segment : 0x7ffdbcd43bac
eesha@DESKTOP-6R05L78:~$ |
```